

Network Security Protocols and Configuration

Introduction

The purpose of this lab was to gain practical experience in configuring and verifying common network security protocols used to protect network communications and remote access. The lab focused on implementing SSL/TLS for secure web communication, SSH for secure remote access, IPsec for VPN communication, and 802.1X using FreeRADIUS for network access control.

The configurations were carried out on a Kali Linux virtual machine using industry-standard tools. After each protocol was configured, tests were performed to verify that the services were working correctly. During the lab, a few configuration issues were encountered and resolved through troubleshooting, providing a better understanding of deploying and maintaining secure network services.

Objective

- Configure secure network services using Apache, SSH, SSL/TLS, IPsec (strongSwan), and FreeRADIUS.
- Verify secure communication and encryption between systems.
- Gain practical experience in implementing and troubleshooting network security protocols.

Materials Used

- Kali Linux
- Apache2 Web Server

- OpenSSH Server (SSH)
- OpenSSL
- strongSwan (IPsec VPN)
- FreeRADIUS
- GNU Nano Text Editor
- Terminal
- Internet Connection

Part 1: Configure SSL/TLS (HTTPS)

Step 1: Install Apache

Commands used:

`sudo apt update` (Update your package list)

`sudo apt install apache2 -y` (Install Apache Web Server)

`sudo systemctl start apache2` (Start the Apache service)

`sudo systemctl status apache2` (checks if Apache is running)



```
(kali@kali)-[~]
└─$ sudo apt update
Hit:1 http://http.kali.org/kali kali-rolling InRelease
All packages are up to date.

(kali@kali)-[~]
└─$ sudo apt install apache2 -y
apache2 is already the newest version (2.4.68-1).
apache2 set to manually installed.
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0

(kali@kali)-[~]
└─$ sudo systemctl start apache2

(kali@kali)-[~]
└─$ sudo systemctl status apache2
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; disabled; preset: disabled)
   Active: active (running) since Fri 2026-07-10 00:13:46 EDT; 21s ago
 Invocation: 2aaefb5b50fd4844825f22c8f59c0338
    Docs: https://httpd.apache.org/docs/2.4/
   Main PID: 65489 (apache2)
  Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
     Tasks: 6 (limit: 2069)
    Memory: 25.7M (peak: 26.3M)
       CPU: 193ms
   CGroup: /system.slice/apache2.service
           └─65489 /usr/sbin/apache2 -k start -DFOREGROUND
             └─65492 /usr/sbin/apache2 -k start -DFOREGROUND
               └─65493 /usr/sbin/apache2 -k start -DFOREGROUND
                 └─65494 /usr/sbin/apache2 -k start -DFOREGROUND
                   └─65495 /usr/sbin/apache2 -k start -DFOREGROUND
                     └─65496 /usr/sbin/apache2 -k start -DFOREGROUND

Jul 10 00:13:46 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Jul 10 00:13:46 kali apachectl[65489]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
Jul 10 00:13:46 kali systemd[1]: Started apache2.service - The Apache HTTP Server.

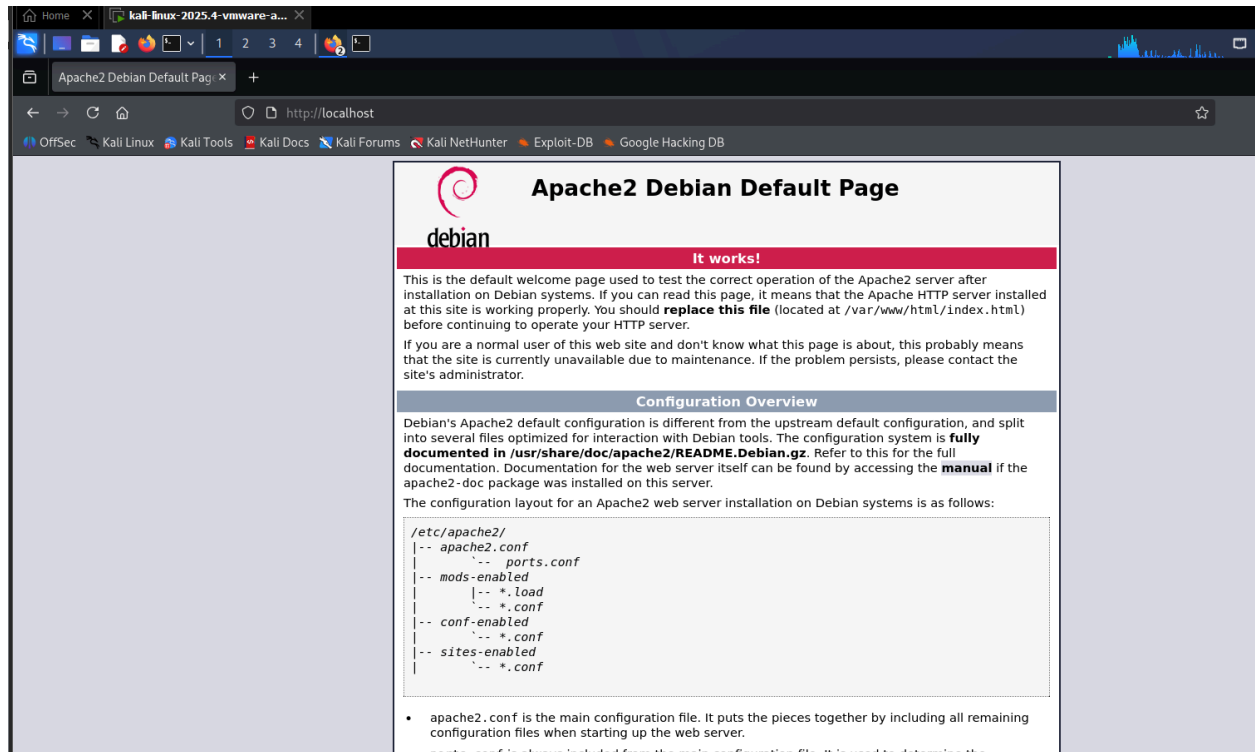
(kali@kali)-[~]
└─$
```

- All packages are up-to-date
- Apache is already the newest version.
- Apache started successfully and running.

Step 1.2 Test Apache in a web browser

Command Used:

`http://localhost`



Apache was installed and verified successfully. The service was running, and the default Apache page loaded at <http://localhost>.

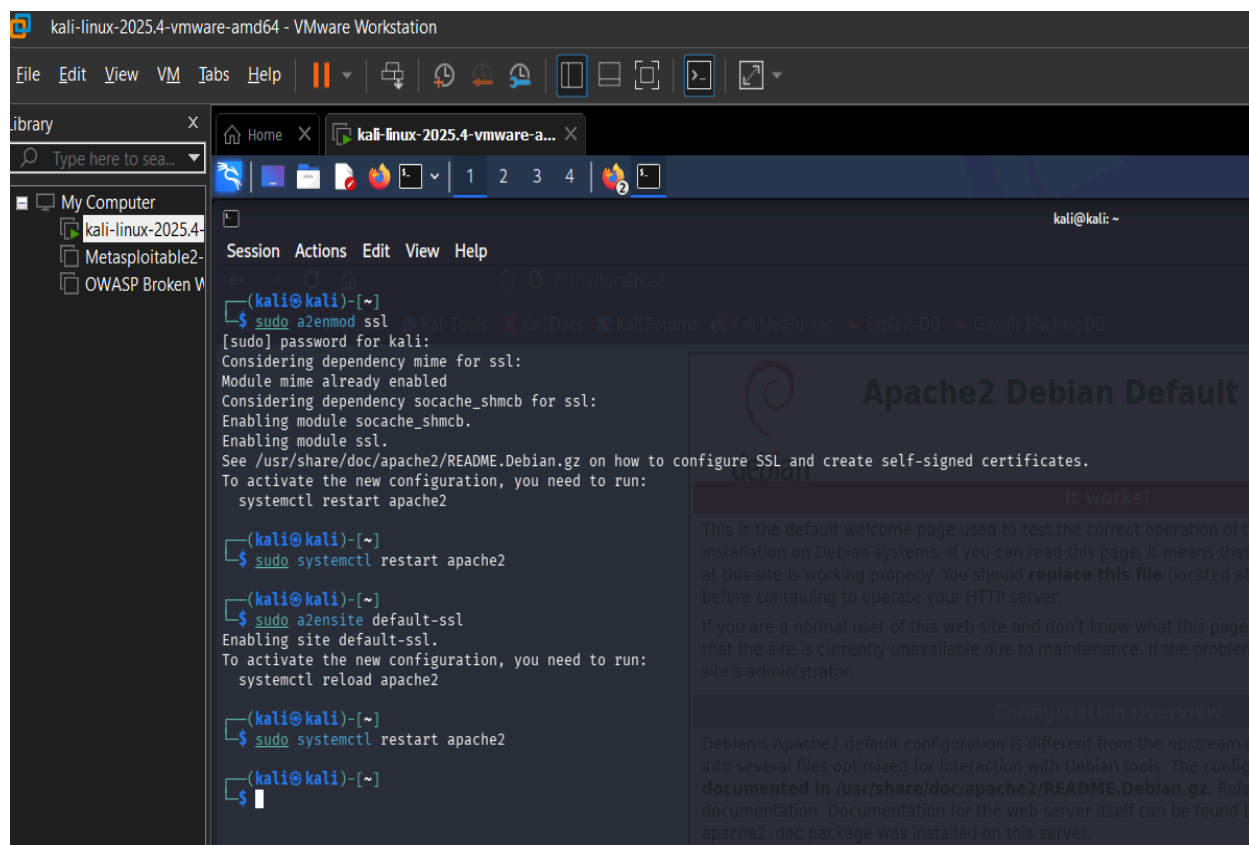
Step 2: Enable SSL Module & Enable SSL site

Commands Used:

`sudo a2ensite default-ssl` (Enables SSL)

`sudo a2ensite default-ssl` (Enables SSL site)

`sudo systemctl restart apache2` (Restart Apache)



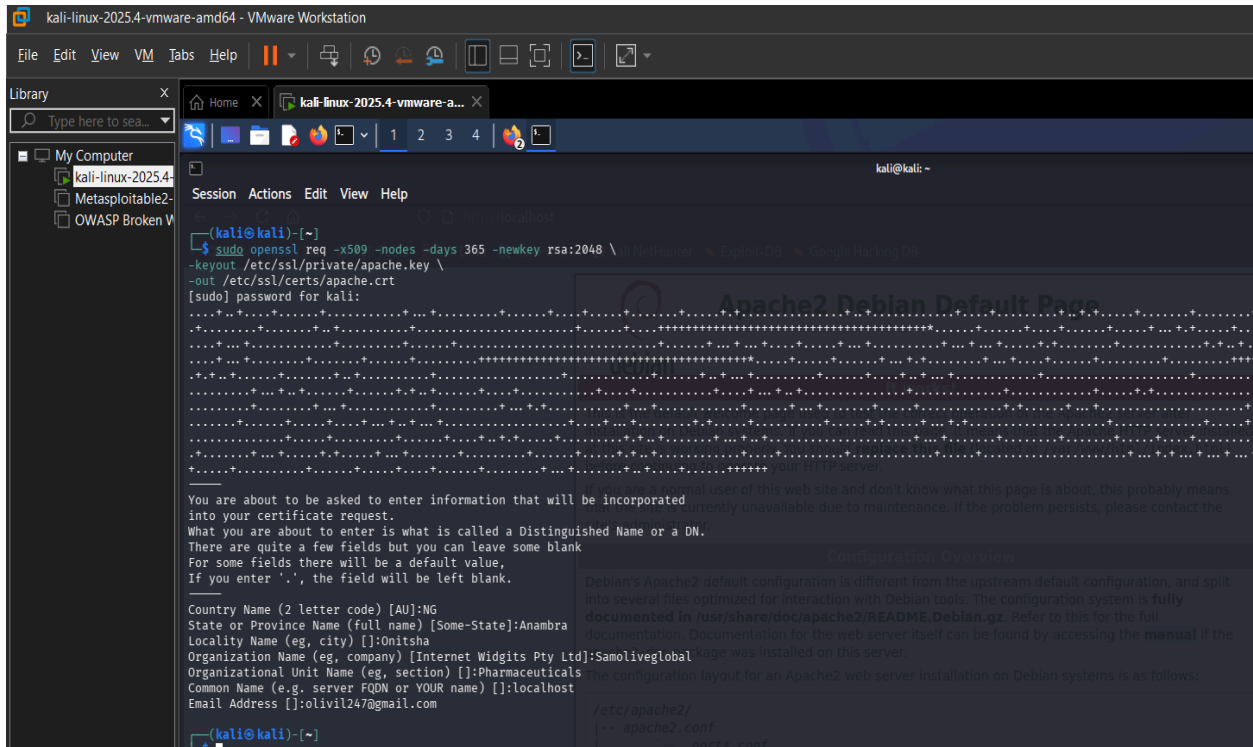
The SSL module was enabled in Apache using `a2enmod ssl`. The default SSL site was activated with `a2ensite default-ssl`, and Apache was restarted to apply the changes. The SSL configuration was enabled successfully.

Step 3: Create a Self-Signed Certificate

Commands Used:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
```

-keyout /etc/ssl/private/apache.key \
-out /etc/ssl/certs/apache.crt



```
kali@kali:~$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/ssl/private/apache.key \
-out /etc/ssl/certs/apache.crt
[sudo] password for kali:
.....
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:NG
State or Province Name (full name) [Some-State]:Anambra
Locality Name (eg, city) []:Onitsha
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Samoliveglobal
Organizational Unit Name (eg, section) []:Pharmaceuticals
Common Name (e.g. server FQDN or YOUR name) []:localhost
Email Address []:olivil247@gmail.com
-----
kali@kali:~$
```

A self-signed SSL certificate was successfully created using OpenSSL. The certificate and private key were generated and stored for Apache to use in secure HTTPS communication.

Step 4: Configure Apache to Use the SSL Certificate

Command Used to Open the SSL configuration file:

```
sudo nano /etc/apache2/sites-available/default-ssl.conf
```

Found these lines: SSLCertificateFile

/etc/ssl/certs/ssl-cert-snakeoil.pem SSLCertificateKeyFile

/etc/ssl/private/ssl-cert-snakeoil.key

Replace with: SSLCertificateFile /etc/ssl/certs/apache.crt

SSLCertificateKeyFile /etc/ssl/private/apache.key

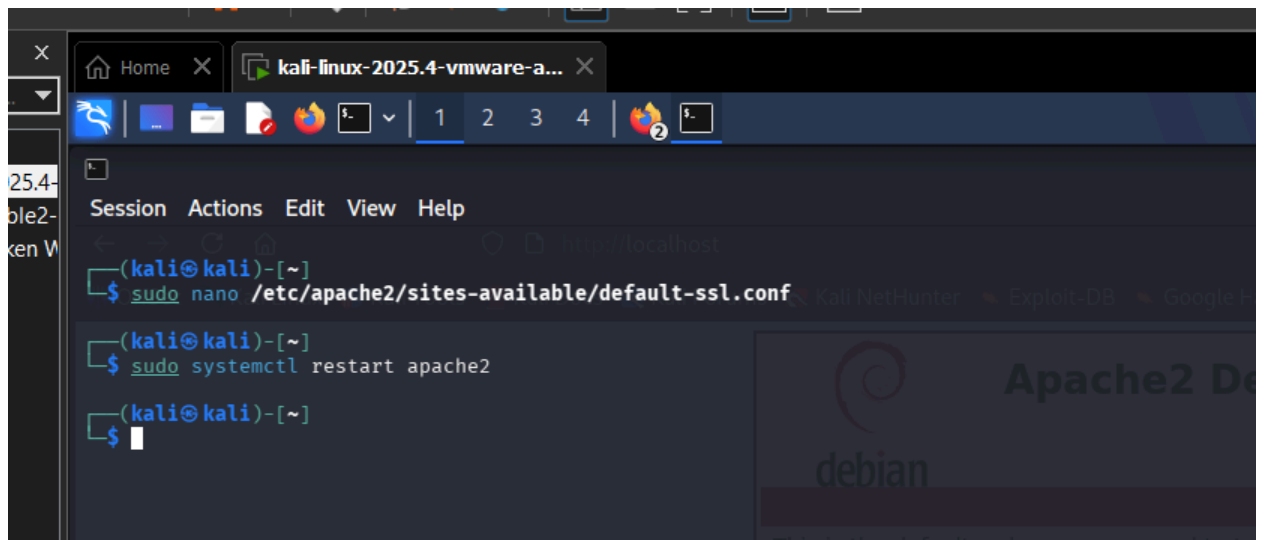
Saved and exit:

Ctrl + O, Enter

Ctrl + X

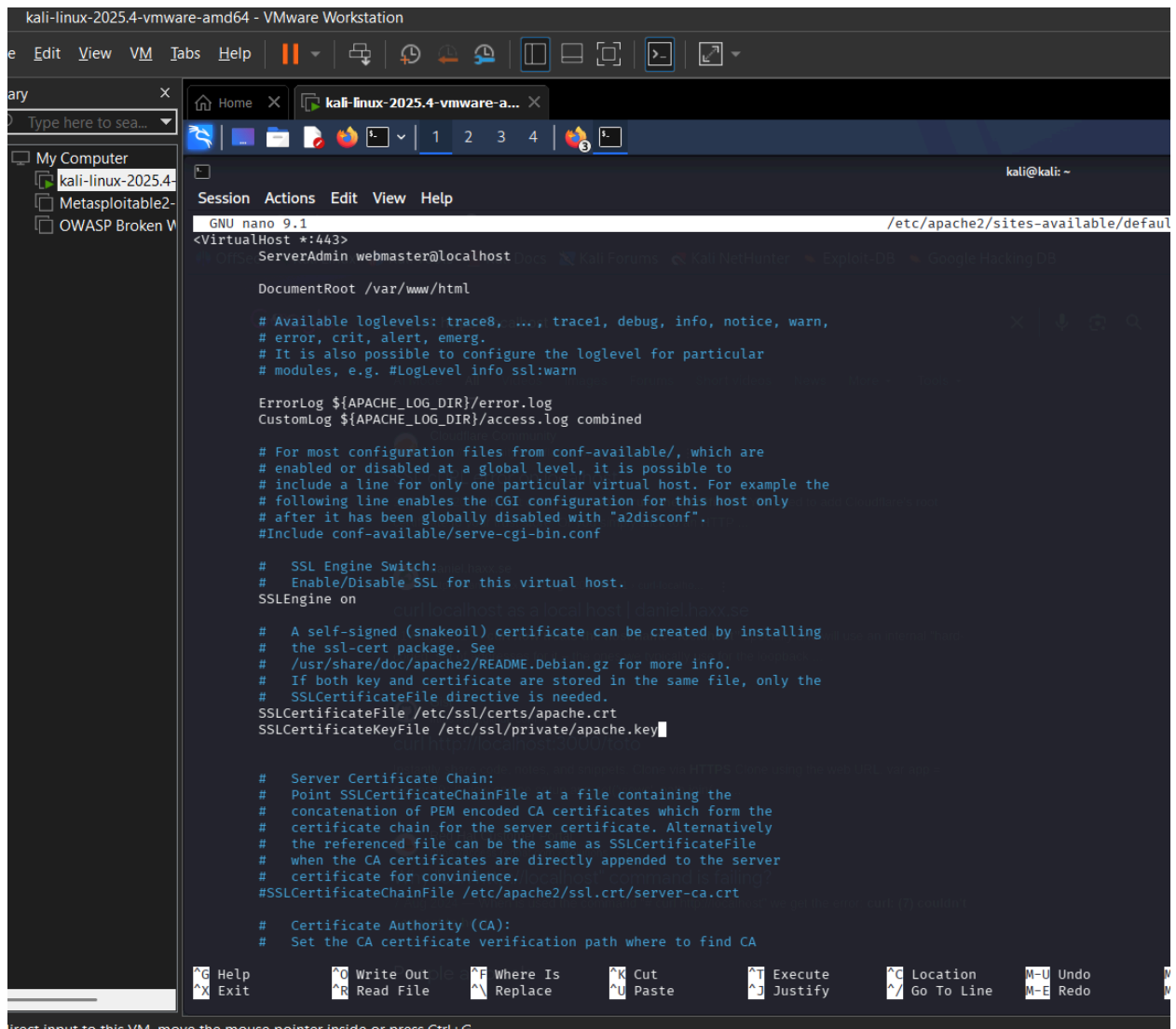
Restart Apache:

sudo systemctl restart apache2



```
(kali㉿kali)-[~]  
$ sudo nano /etc/apache2/sites-available/default-ssl.conf  
  
(kali㉿kali)-[~]  
$ sudo systemctl restart apache2  
  
(kali㉿kali)-[~]  
$
```

Apache was configured to use the newly created self-signed SSL certificate and private key. The configuration was saved, and Apache was restarted successfully.



The screenshot shows a Kali Linux virtual machine running in VMware Workstation. The terminal window is open, displaying the configuration of the Apache2 default site in the nano editor. The configuration file is located at `/etc/apache2/sites-available/default.conf`. The configuration includes the following settings:

```
ServerAdmin webmaster@localhost
DocumentRoot /var/www/html

# Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
# error, crit, alert, emerg.
# It is also possible to configure the loglevel for particular
# modules, e.g. #LogLevel info ssl:warn

ErrorLog ${APACHE_LOG_DIR}/error.log
CustomLog ${APACHE_LOG_DIR}/access.log combined

# For most configuration files from conf-available/, which are
# enabled or disabled at a global level, it is possible to
# include a line for only one particular virtual host. For example the
# following line enables the CGI configuration for this host only
# after it has been globally disabled with "a2disconf".
#Include conf-available/serve-cgi-bin.conf

# SSL Engine Switch:
# Enable/Disable SSL for this virtual host.
SSLEngine on

# A self-signed (snakeoil) certificate can be created by installing
# the ssl-cert package. See
# /usr/share/doc/apache2/README.Debian.gz for more info.
# If both key and certificate are stored in the same file, only the
# SSLCertificateFile directive is needed.
SSLCertificateFile /etc/ssl/certs/apache.crt
SSLCertificateKeyFile /etc/ssl/private/apache.key

# Server Certificate Chain:
# Point SSLCertificateChainFile at a file containing the
# concatenation of PEM encoded CA certificates which form the
# certificate chain for the server certificate. Alternatively
# the referenced file can be the same as SSLCertificateFile
# when the CA certificates are directly appended to the server
# certificate for convenience.
#SSLCertificateChainFile /etc/apache2/ssl.crt/server-ca.crt

# Certificate Authority (CA):
# Set the CA certificate verification path where to find CA
```

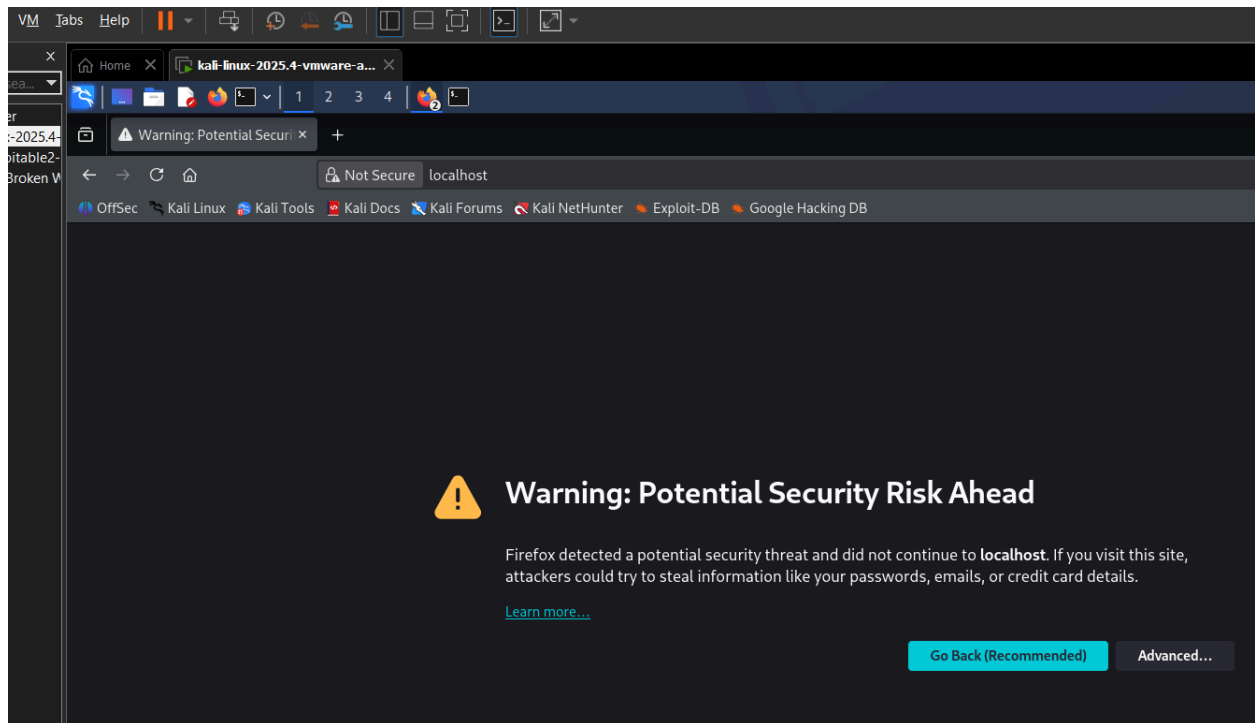
The terminal window also shows the prompt `kali@kali: ~` and the file manager sidebar on the left.

Apache was configured to use the newly created self-signed SSL certificate and private key. The configuration was saved, and Apache was restarted successfully.

Step 5: Test HTTPS

Command Used:

<https://localhost>

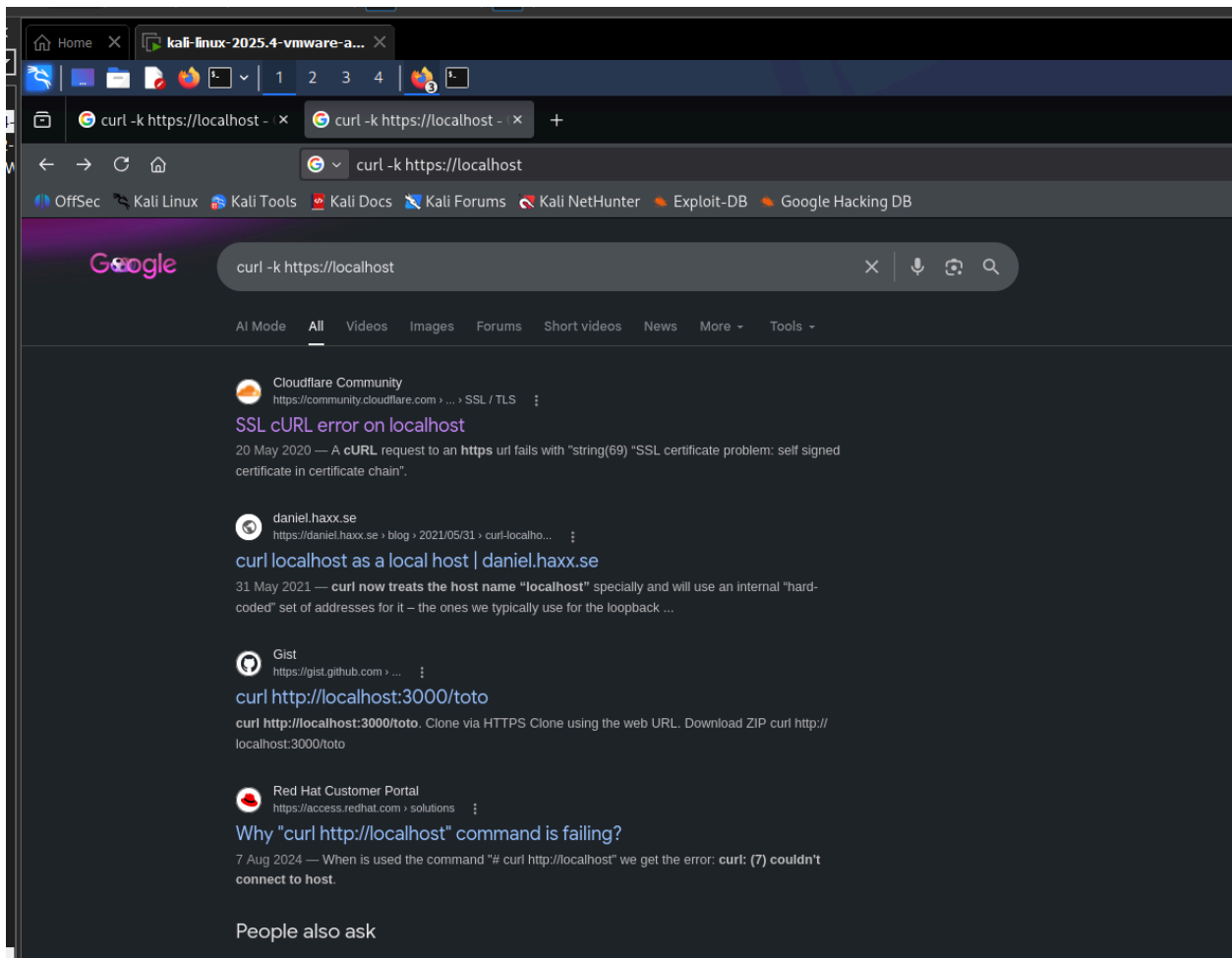


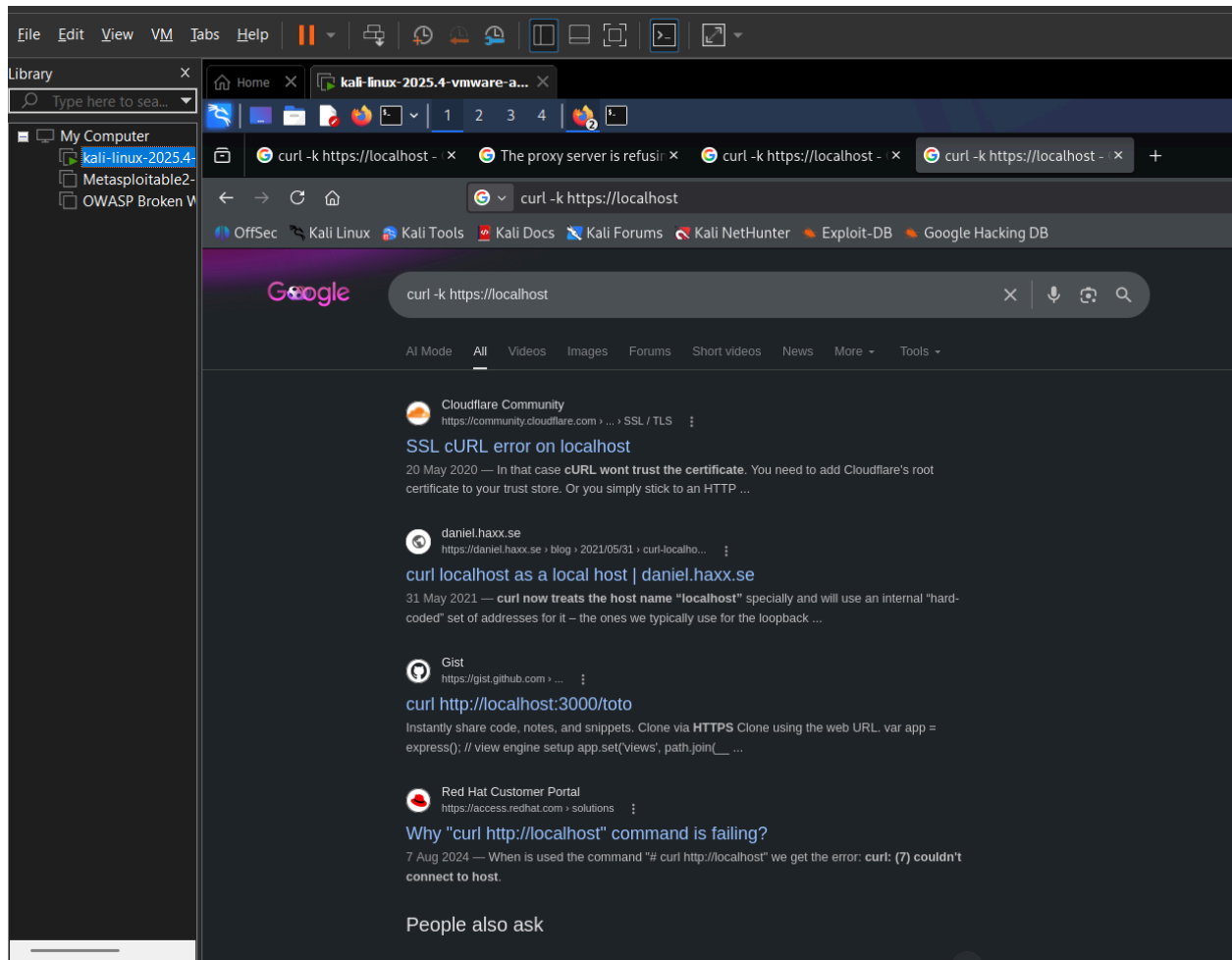
HTTPS was successfully tested by accessing <https://localhost>. The browser displayed a security warning due to the self-signed certificate, which was accepted. The Apache page loaded securely over HTTPS.

Step 6: Verify HTTPS Using curl

Command Used:

`curl -k https://localhost`





HTTPS was successfully verified using `curl`. The web server responded over a secure HTTPS connection, confirming that the SSL/TLS configuration was working correctly.

Part 2: Configure SSH for Secure Remote Access

Step 1: Install and Verify SSH Server

Command Used:

Install the SSH server:

```
sudo apt install openssh-server -y
```

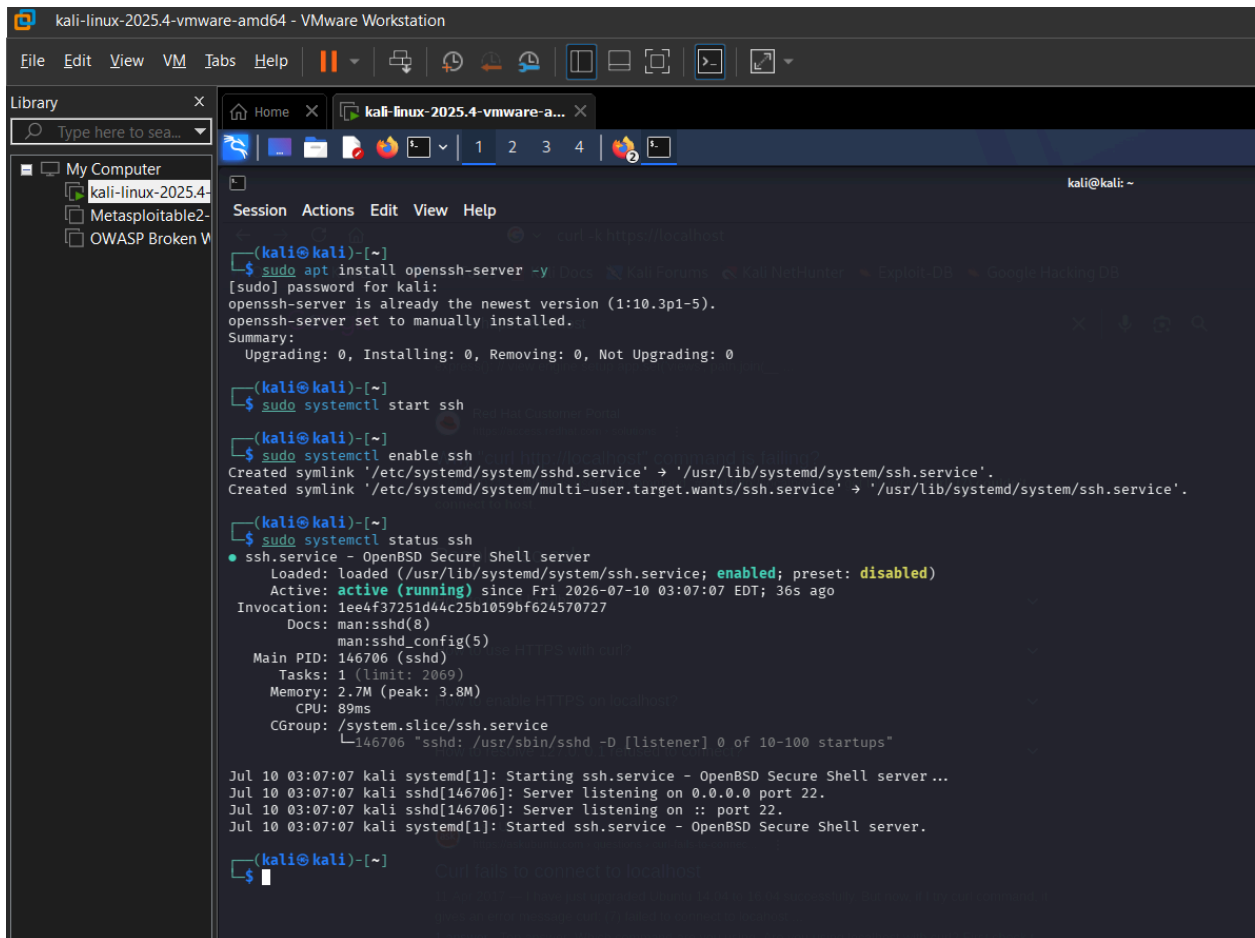
Start and enable the service:

```
sudo systemctl start ssh
```

```
sudo systemctl enable ssh
```

Verify it is running:

```
sudo systemctl status ssh
```



```
kali-linux-2025.4-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
Library
Type here to search
My Computer
kali-linux-2025.4
Metasploitable2
OWASP Broken VMs

kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ sudo apt install openssh-server -y
[sudo] password for kali:
openssh-server is already the newest version (1:10.3p1-5).
openssh-server set to manually installed.
Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0

(kali@kali)-[~]
$ sudo systemctl start ssh

(kali@kali)-[~]
$ sudo systemctl enable ssh
Created symlink '/etc/systemd/system/ssh.service' → '/usr/lib/systemd/system/ssh.service'.
Created symlink '/etc/systemd/system/multi-user.target.wants/ssh.service' → '/usr/lib/systemd/system/ssh.service'.

(kali@kali)-[~]
$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-07-10 03:07:07 EDT; 36s ago
   Invocation: 1ee4f37251d44c25b1059bf624570727
   Docs: man:sshd(8)
        man:sshd_config(5)
   Main PID: 146706 (sshd)
   Tasks: 1 (limit: 2069)
   Memory: 2.7M (peak: 3.8M)
   CPU: 89ms
   CGroup: /system.slice/ssh.service
           └─146706 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jul 10 03:07:07 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server ...
Jul 10 03:07:07 kali sshd[146706]: Server listening on 0.0.0.0 port 22.
Jul 10 03:07:07 kali sshd[146706]: Server listening on :: port 22.
Jul 10 03:07:07 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.

(kali@kali)-[~]
$
```

The OpenSSH Server was verified to be installed and running successfully on the Kali Linux system, enabling secure remote access.

Step 2: Change the Default SSH Port

Commands Used & Method:

`sudo nano /etc/ssh/sshd_config` (Open the SSH configuration file)

Found: #Port 22

Change it to: Port 2222

Saved and exit:

Ctrl + O, Enter

Ctrl + X

Restart SSH:

`sudo systemctl restart ssh`

Then run:

`sudo systemctl status ssh`

```
prary x
Type here to sea... x
kali-linux-2025.4-vmware-a... x
1 2 3 4
kali@kali: ~
Session Actions Edit View Help
GNU nano 9.1 /etc/ssh/sshd_

# This is the sshd server system-wide configuration file. See NetHunter Exploit-DB Google Hacking DB
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

#Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key/so ask
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH

#LogLevel INFO

# Authentication:

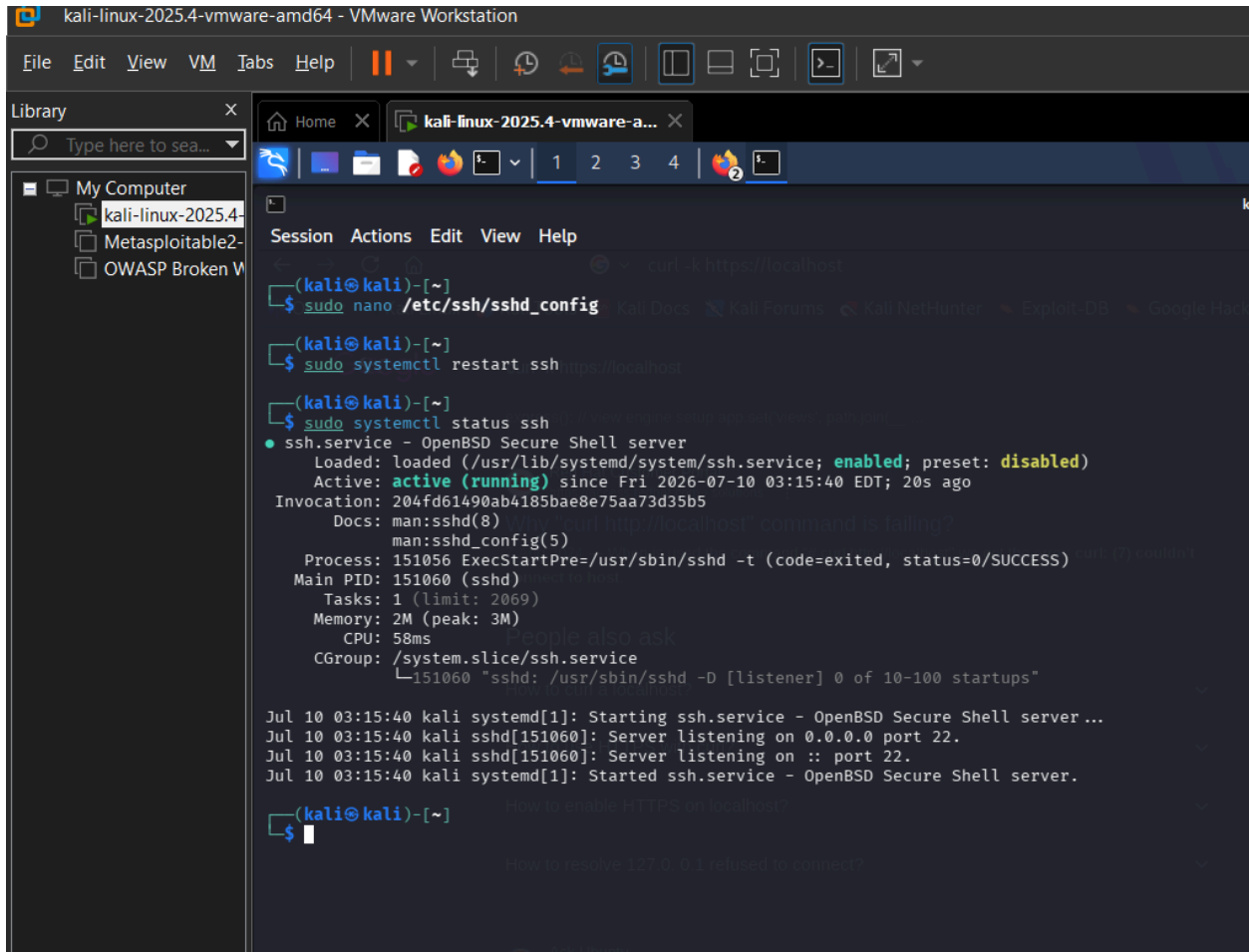
#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes No
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^N Replace     ^U Paste      ^J Justify
^_          ^_          ^_          ^_          ^_          ^_
^C Location  M-U Undo
^_/ Go To Line M-E Redo
```



```
GNU nano 9.1 /etc/ss

# This is the sshd server system-wide configuration file. See NetHunter, Exploit-DB, Google Hacking DB
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes No
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-
```

The default SSH port was changed from 22 to 2222, and the SSH service was restarted successfully. The new configuration was verified and is active

Step 3: Disable Root Login

Commands Used & Method:

`sudo nano /etc/ssh/sshd_config` (Open the SSH configuration file)

Find:

`#PermitRootLogin yes`

Changed it to:

`PermitRootLogin no`

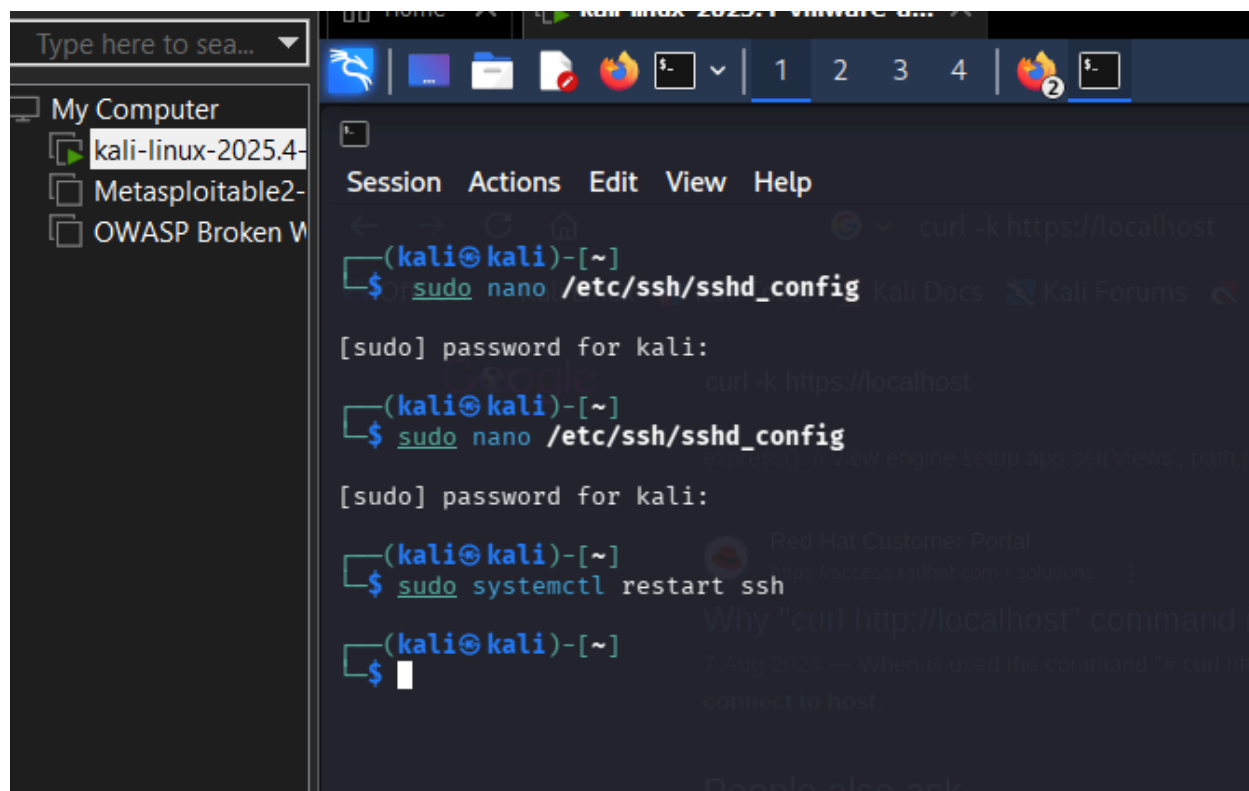
Saved and exit:

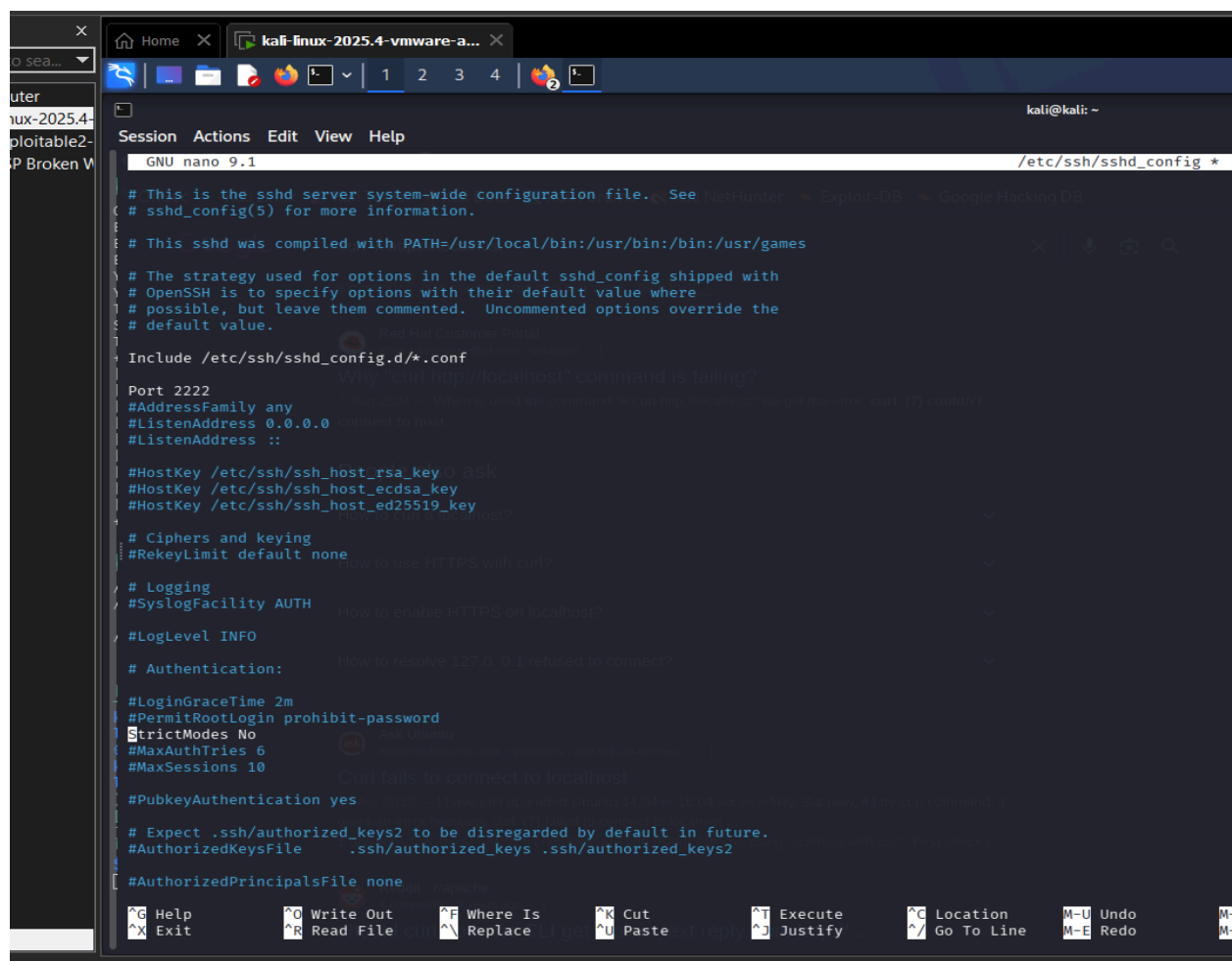
Ctrl + O, Enter

Ctrl + X

then restart SSH:

`sudo systemctl restart ssh`





```
GNU nano 9.1 /etc/ssh/sshd_config *
# This is the sshd server system-wide configuration file. See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.

# Include /etc/ssh/sshd_config.d/*.conf

#Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
StrictModes No
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none
```

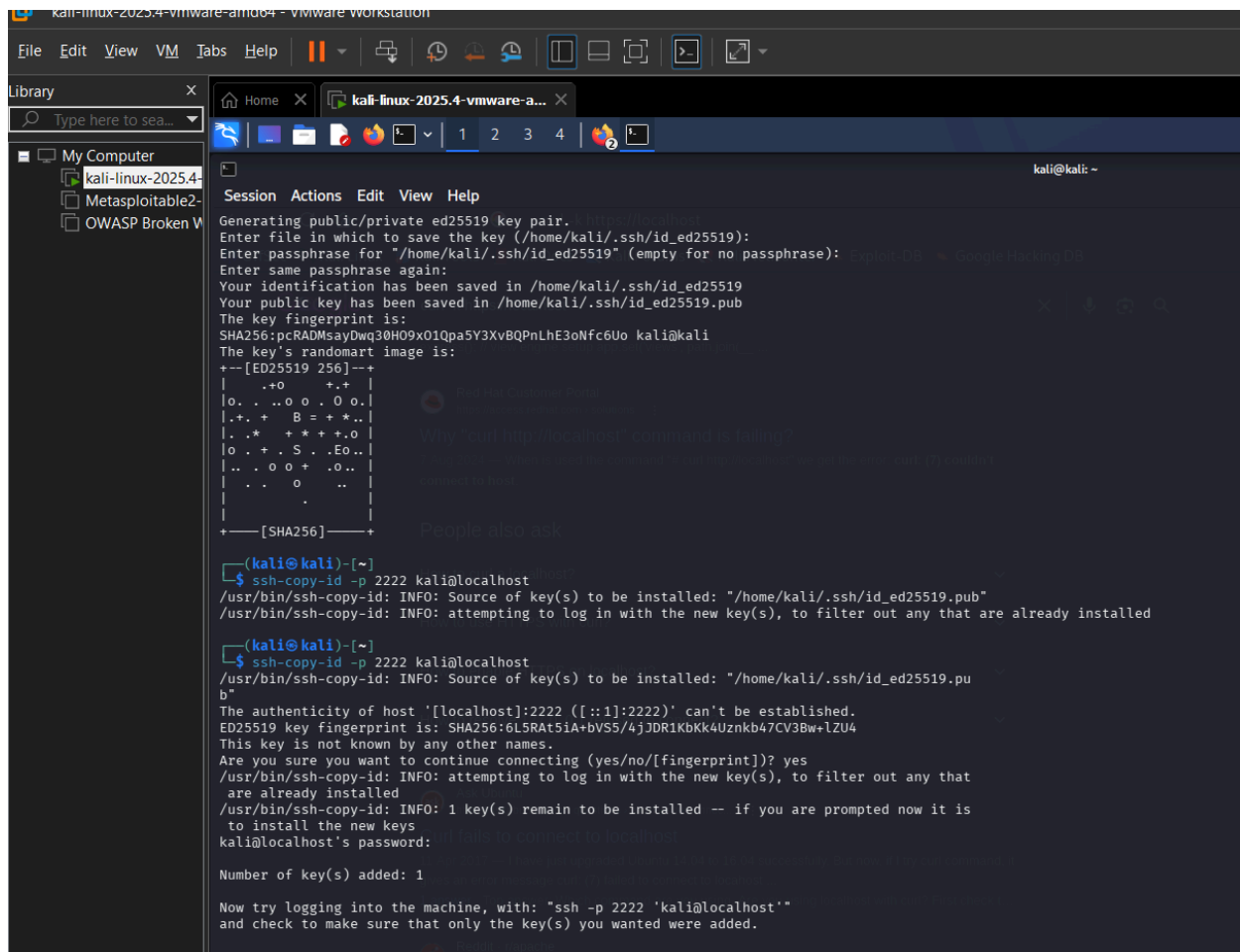
Root login was disabled in the SSH configuration, and the SSH service was restarted successfully to apply the security setting.

Step 4: Configure Public Key Authentication

Commands Used:

ssh-keygen (Generate an SSH key pair)

ssh-copy-id -p 2222 kali@localhost (Copy the Public Key)



```
kali@kali: ~$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kali/.ssh/id_ed25519):
Enter passphrase for "/home/kali/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_ed25519
Your public key has been saved in /home/kali/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:pcRADMsayDwq30H09x01Qpa5Y3XvBQPnLhE3oNfc6Uo kali@kali
The key's randomart image is:
+--[ED25519 256]--+
|      .+      |
| o. . . o . o o |
| .+ . B = + .+ |
| .* + + + .+ o |
| o . + . S . . Eo..|
| .. . o o + . o ..|
| . . . . . |
|      .      |
+---[SHA256]-----+

(kali@kali)-[~]
$ ssh-copy-id -p 2222 kali@localhost
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/kali/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed

(kali@kali)-[~]
$ ssh-copy-id -p 2222 kali@localhost
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/kali/.ssh/id_ed25519.pub"
The authenticity of host '[localhost]:2222 ([::1]:2222)' can't be established.
ED25519 key fingerprint is: SHA256:6L5RAt5iA+bVS5/4jJDR1KbKk4Uznkb47CV3Bw+LZU4
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
kali@localhost's password:
Number of key(s) added: 1

Now try logging into the machine, with: "ssh -p 2222 'kali@localhost'"
and check to make sure that only the key(s) you wanted were added.
```

An SSH public/private key pair was generated successfully using ssh-keygen for secure key-based authentication.

The SSH public key was successfully copied to the local SSH server using ssh-copy-id. This enabled key-based authentication for secure remote access.

Test SSH Login
ssh -p 2222 kali@localhost

```
(kali㉿kali)-[~]  
$ ssh -p 2222 kali@localhost  
Linux kali 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1+kali1 (2026-05-05) x86_64  
  
The programs included with the Kali GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
You have mail.  
zsh: corrupt history file /home/kali/.zsh_history  
(kali㉿kali)-[~]  
$
```

SSH login was successfully tested using port 2222. The connection was established successfully, confirming that secure remote access was working.

Problem Encountered

SSH connections on port 2222 were refused because the SSH service was still listening on the default port 22.

```
ry
X
Type here to sea...
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken W

kali-linux-2025.4-vmware-a... X
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447 448 449 450 451 452 453 454 455 456 457 458 459 460 461 462 463 464 465 466 467 468 469 470 471 472 473 474 475 476 477 478 479 480 481 482 483 484 485 486 487 488 489 490 491 492 493 494 495 496 497 498 499 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841 842 843 844 845 846 847 848 849 850 851 852 853 854 855 856 857 858 859 860 861 862 863 864 865 866 867 868 869 870 871 872 873 874 875 876 877 878 879 880 881 882 883 884 885 886 887 888 889 890 891 892 893 894 895 896 897 898 899 900 901 902 903 904 905 906 907 908 909 910 911 912 913 914 915 916 917 918 919 920 921 922 923 924 925 926 927 928 929 930 931 932 933 934 935 936 937 938 939 940 941 942 943 944 945 946 947 948 949 950 951 952 953 954 955 956 957 958 959 960 961 962 963 964 965 966 967 968 969 970 971 972 973 974 975 976 977 978 979 980 981 982 983 984 985 986 987 988 989 990 991 992 993 994 995 996 997 998 999 1000

Session Actions Edit View Help

(kali@kali)-[~]
$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-07-10 04:36:08 EDT; 19s ago
   Invocation: 0967874608bd4f55a0bb05ae876db56c
     Docs: man:sshd(8)
           man:sshd_config(5)
   Process: 188565 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 188570 (sshd)
     Tasks: 1 (limit: 2069)
    Memory: 3M (peak: 4M)
       CPU: 75ms
    CGroup: /system.slice/ssh.service
            └─188570 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Jul 10 04:36:08 kali systemd[1]: Starting ssh.service - OpenBSD Secure Shell server.
Jul 10 04:36:08 kali sshd[188570]: Server listening on 0.0.0.0 port 22.
Jul 10 04:36:08 kali sshd[188570]: Server listening on :: port 22.
Jul 10 04:36:08 kali systemd[1]: Started ssh.service - OpenBSD Secure Shell server.

(kali@kali)-[~]
$ sudo sshd -T | grep port
port 22
gatewayports no

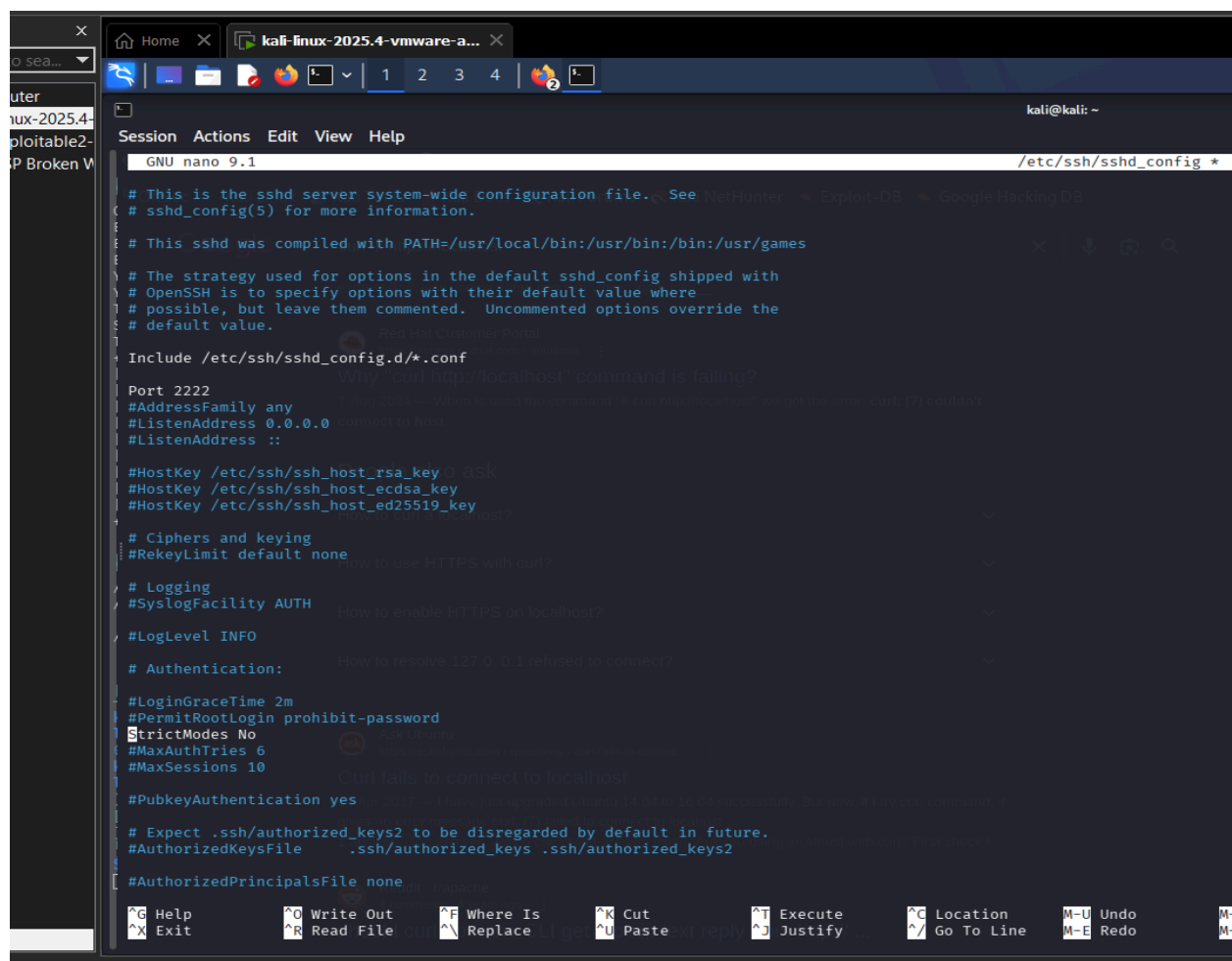
(kali@kali)-[~]
$ ls /etc/ssh/sshd_config.d/
20-systemd-userdb.conf

(kali@kali)-[~]
$ grep -R "Port" /etc/ssh/
grep: /etc/ssh/ssh_host_ed25519_key: Permission denied
grep: /etc/ssh/ssh_host_ecdsa_key: Permission denied
/etc/ssh/ssh_config:# Port 22
grep: /etc/ssh/ssh_host_rsa_key: Permission denied
/etc/ssh/sshd_config:#Port 2222
/etc/ssh/sshd_config:#GatewayPorts no

(kali@kali)-[~]
$ sudo nano /etc/ssh/sshd_config

(kali@kali)-[~]
$ sudo systemctl restart ssh

(kali@kali)-[~]
$ sudo sshd -T | grep port
port 2222
```



```
GNU nano 9.1 /etc/ssh/sshd_config *
# This is the sshd server system-wide configuration file. See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.

# Include /etc/ssh/sshd_config.d/*.conf

Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes No
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded by default in future.
#AuthorizedKeysFile .ssh/authorized_keys .ssh/authorized_keys2

#AuthorizedPrincipalsFile none
```

Troubleshooting Steps

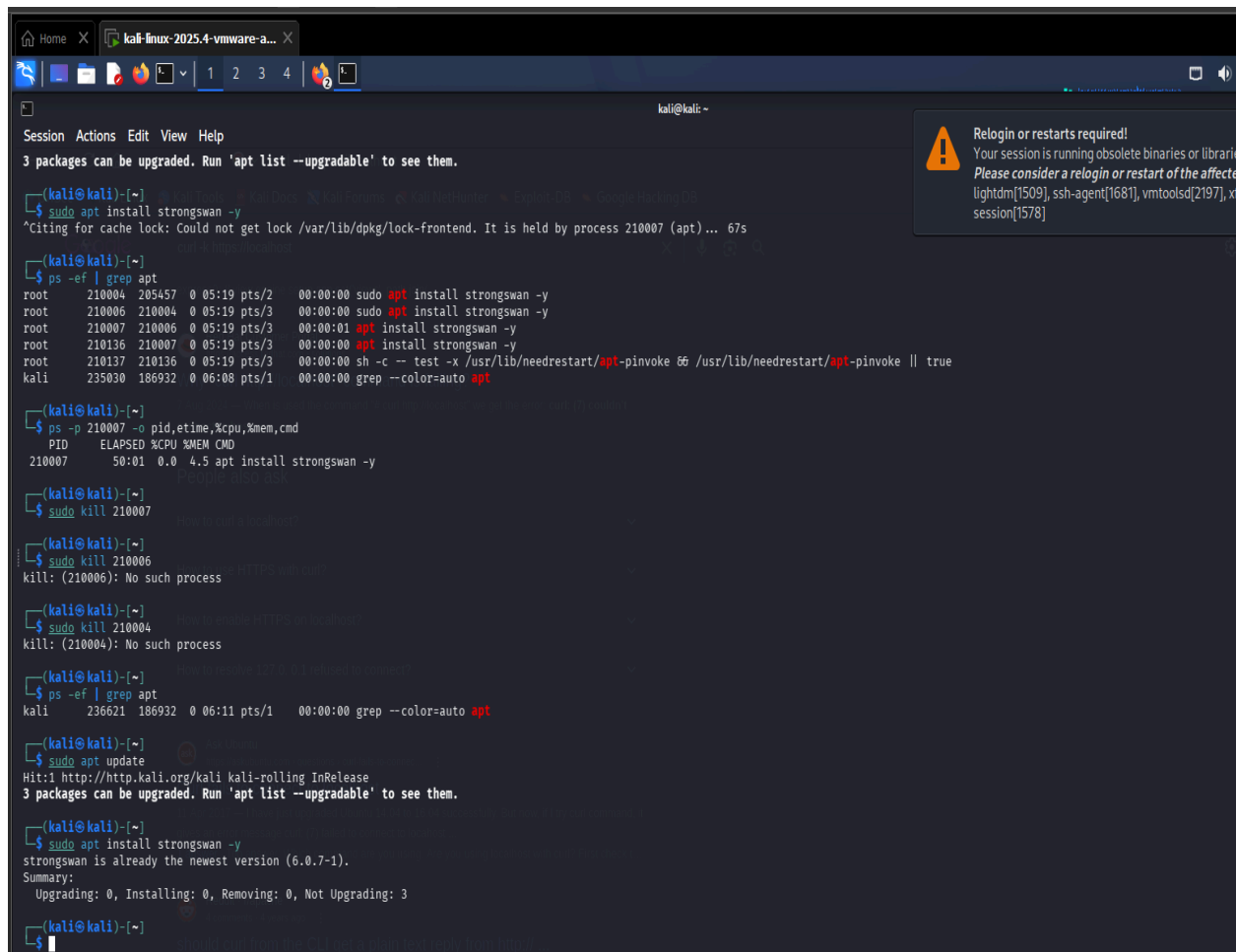
The SSH configuration was checked and it was discovered that the Port 2222 line was commented out (#Port 2222). The # was removed, the SSH service was restarted, and the configuration was verified. SSH then listened successfully on port 2222.

Part 3: Implement IPsec VPN

Step 1: Install StrongSwan

sudo apt update

sudo apt install strongswan -y



```
(kali@kali)-[~]
└─$ sudo apt install strongswan -y
^Citing for cache lock: Could not get lock /var/lib/dpkg/lock-frontent. It is held by process 210007 (apt) ... 67s

(kali@kali)-[~]
└─$ ps -ef | grep apt
root      210004  205457  0 05:19 pts/2    00:00:00 sudo apt install strongswan -y
root      210006  210004  0 05:19 pts/3    00:00:00 sudo apt install strongswan -y
root      210007  210006  0 05:19 pts/3    00:00:01 apt install strongswan -y
root      210136  210007  0 05:19 pts/3    00:00:00 apt install strongswan -y
root      210137  210136  0 05:19 pts/3    00:00:00 sh -c -- test -x /usr/lib/needrestart/apt-pinvoke && /usr/lib/needrestart/apt-pinvoke || true
kali      235030  186932  0 06:08 pts/1    00:00:00 grep --color=auto apt

(kali@kali)-[~]
└─$ ps -p 210007 -o pid,etime,%cpu,%mem,cmd
PID      ELAPSED %CPU %MEM CMD
210007    50:01  0.0  4.5 apt install strongswan -y

(kali@kali)-[~]
└─$ sudo kill 210007
kill: (210006): No such process

(kali@kali)-[~]
└─$ sudo kill 210006
kill: (210006): No such process

(kali@kali)-[~]
└─$ sudo kill 210004
kill: (210004): No such process

(kali@kali)-[~]
└─$ ps -ef | grep apt
kali      236621  186932  0 06:11 pts/1    00:00:00 grep --color=auto apt

(kali@kali)-[~]
└─$ sudo apt update
Hit:1 http://http.kali.org/kali kali-rolling InRelease
3 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@kali)-[~]
└─$ sudo apt install strongswan -y
strongswan is already the newest version (6.0.7-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 3

(kali@kali)-[~]
└─$
```

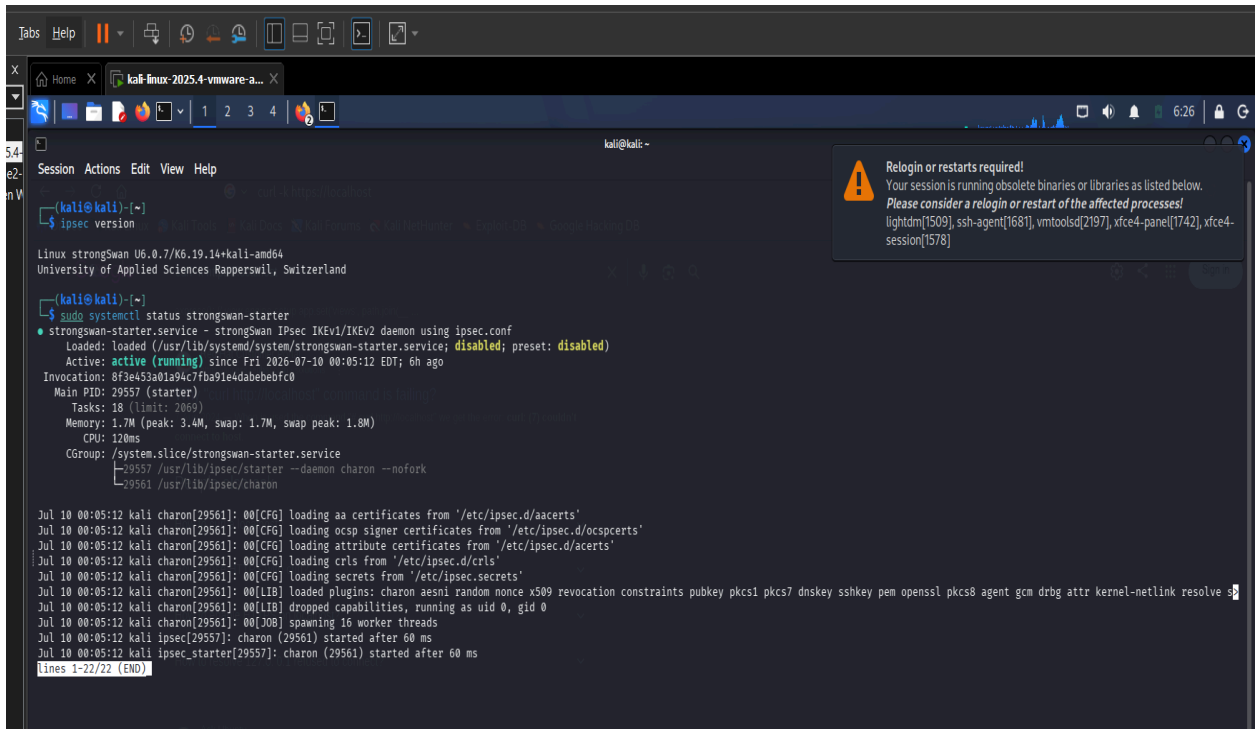
Step 2: Verify the StrongSwan Installation

verify it:

ipsec version

Then check the service status:

sudo systemctl status strongswan-starter



```
kali@kali:~$ ipsec version
Linux strongSwan U6.0.7/K6.19.14+kali-amd64
University of Applied Sciences Rapperswil, Switzerland

kali@kali:~$ sudo systemctl status strongswan-starter
● strongswan-starter.service - strongSwan IPsec IKEv1/IKEv2 daemon using ipsec.conf
   Loaded: loaded (/usr/lib/systemd/system/strongswan-starter.service; disabled; preset: disabled)
   Active: active (running) since Fri 2026-07-10 00:05:12 EDT; 6h ago
     Invocation: 8f3e453a01a94c7fba91e4d4dabebefc0
   Main PID: 29557 (starter)
      Tasks: 18 (limit: 2069)
     Memory: 1.7M (peak: 3.4M, swap: 1.7M, swap peak: 1.8M)
        CPU: 120ms
   CGroup: /system.slice/strongswan-starter.service
           └─29557 /usr/lib/ipsec/starter --daemon charon --nofork
             └─29561 /usr/lib/ipsec/charon

Jul 10 00:05:12 kali charon[29561]: 00[CFG] loading aa certificates from '/etc/ipsec.d/acerts'
Jul 10 00:05:12 kali charon[29561]: 00[CFG] loading ocsip signer certificates from '/etc/ipsec.d/ocspcerts'
Jul 10 00:05:12 kali charon[29561]: 00[CFG] loading attribute certificates from '/etc/ipsec.d/acerts'
Jul 10 00:05:12 kali charon[29561]: 00[CFG] loading crls from '/etc/ipsec.d/crls'
Jul 10 00:05:12 kali charon[29561]: 00[CFG] loading secrets from '/etc/ipsec.secrets'
Jul 10 00:05:12 kali charon[29561]: 00[L10] loaded plugins: charon aesni random nonce x509 revocation constraints pubkey pkcs1 pkcs7 dnskey sshkey pem openssl pkcs8 agent gcm drbg attr kernel-netlink resolve s
Jul 10 00:05:12 kali charon[29561]: 00[L10] dropped capabilities, running as uid 0, gid 0
Jul 10 00:05:12 kali charon[29561]: 00[J08] spawning 16 worker threads
Jul 10 00:05:12 kali ipsec[29557]: charon (29561) started after 60 ms
Jul 10 00:05:12 kali ipsec_starter[29557]: charon (29561) started after 60 ms
lines 1-22/22 (END)
```

StrongSwan was successfully installed on the Kali Linux system. The installation completed successfully and the IPsec VPN software is ready for configuration.

Problem Encountered

The StrongSwan installation became unresponsive and remained stuck for an extended period because an apt process was hanging.

Troubleshooting Steps

1. Checked the running apt processes:

```
ps -ef | grep apt
```

2. Confirmed the process was stuck:

```
ps -p 210007 -o pid,etime,%cpu,%mem,cmd
```

Terminated the stuck processes:

```
sudo kill 210007
```

```
sudo kill 210006
```

3.

```
sudo kill 210004
```

4. Verified the processes had stopped:

```
ps -ef | grep apt
```

Updated the package list and reinstalled StrongSwan:

```
sudo apt update
```

5.

```
sudo apt install strongswan -y
```

Outcome: The installation completed successfully after restarting the process.

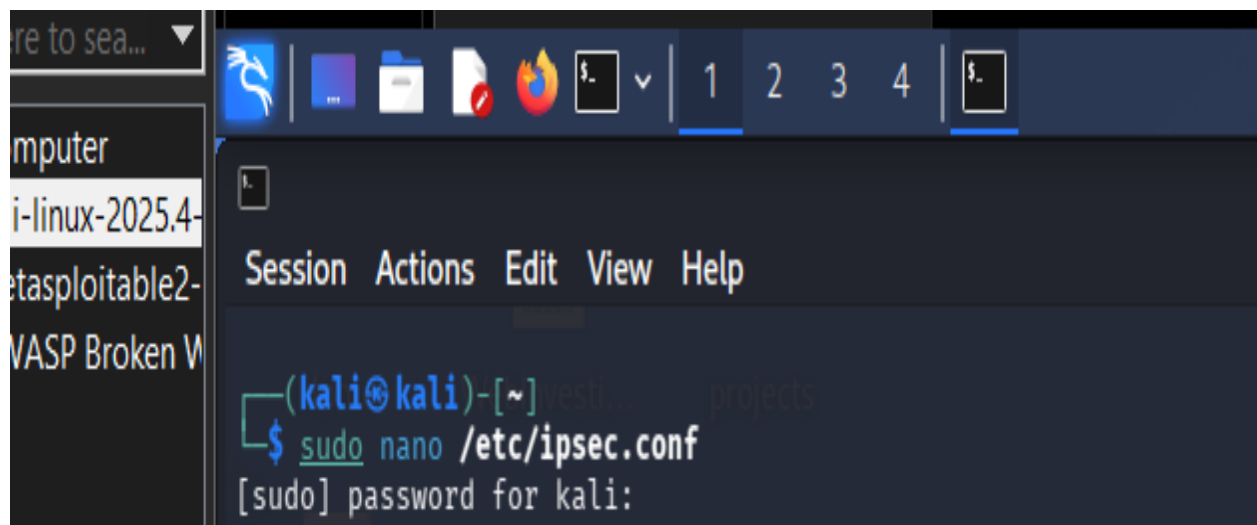
Step 3: Configure IPsec

Open the IPsec configuration file:

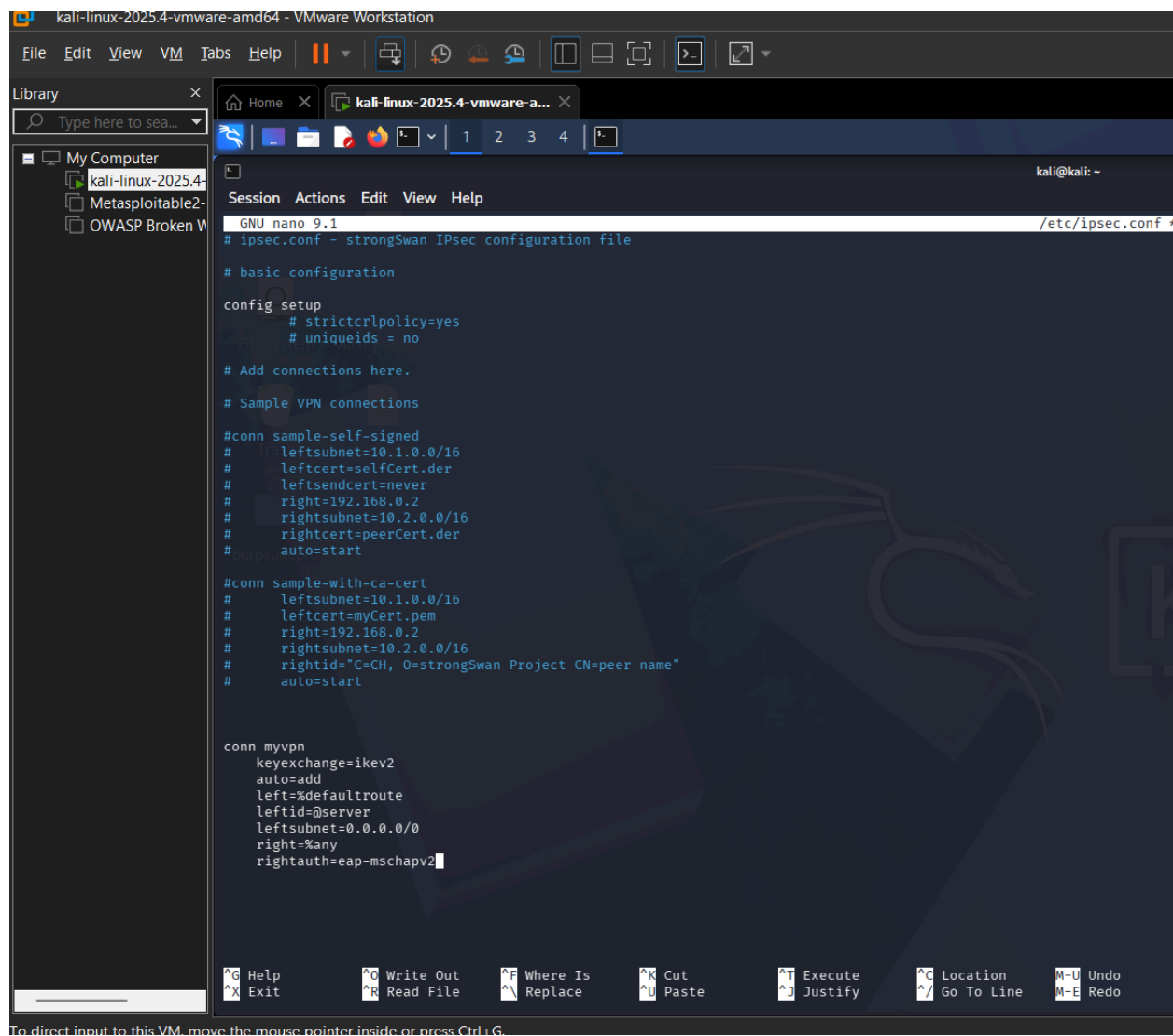
```
sudo nano /etc/ipsec.conf
```

Append with config setup:

```
conn myvpn
    keyexchange=ikev2
    auto=add
    left=%defaultroute
    leftid=@server
    leftsubnet=0.0.0.0/0
    right=%any
    rightauth=eap-mschapv2
```



```
(kali@kali)-[~]
$ sudo nano /etc/ipsec.conf
[sudo] password for kali:
```



The screenshot shows a Kali Linux virtual machine running in VMware Workstation. The terminal window displays the nano 9.1 editor editing the file /etc/ipsec.conf. The file content is as follows:

```
GNU nano 9.1 /etc/ipsec.conf
# ipsec.conf - strongSwan IPsec configuration file

# basic configuration

config_setup
    # strictctrlpolicy=yes
    # uniqueids = no

# Add connections here.

# Sample VPN connections

#conn sample-self-signed
#    leftsubnet=10.1.0.0/16
#    leftcert=selfCert.der
#    leftsendcert=never
#    right=192.168.0.2
#    rightsubnet=10.2.0.0/16
#    rightcert=peerCert.der
#    auto=start

#conn sample-with-ca-cert
#    leftsubnet=10.1.0.0/16
#    leftcert=myCert.pem
#    right=192.168.0.2
#    rightsubnet=10.2.0.0/16
#    rightid="C=CH, O=strongSwan Project CN=peer name"
#    auto=start

conn myvpn
    keyexchange=ikev2
    auto=add
    left=%defaulttroute
    leftid=@server
    leftsubnet=0.0.0.0/0
    right=%any
    rightauth=eap-mschapv2
```

The bottom of the window shows the nano editor's status bar with various shortcuts: ^G Help, ^X Exit, ^O Write Out, ^R Read File, ^F Where Is, ^\ Replace, ^K Cut, ^U Paste, ^T Execute, ^J Justify, ^_ Location, ^_ Go To Line, M-U Undo, M-E Redo.

Save and exit:

Ctrl + O, Enter

Ctrl + X

Restart the IPsec service:

sudo systemctl restart strongswan-starter

Verify the IPsec Configuration:

```
sudo ipsec status
```

Then check the loaded connections:

```
sudo ipsec statusall
```

```
(kali@kali)-[~]
$ sudo systemctl restart strongswan-starter

(kali@kali)-[~]
$ sudo ipsec status
[sudo] password for kali:
Security Associations (0 up, 0 connecting):
    none

(kali@kali)-[~]
$ sudo ipsec statusall
Status of IKE charon daemon (strongSwan 6.0.7, Linux 7.0.12+kali-amd64, x86_64):
uptime: 37 minutes, since Jul 10 06:37:34 2026
malloc: sbrk 2838528, mmap 0, used 651680, free 2186848
worker threads: 11 of 16 idle, 5/0/0/0 working, job queue: 0/0/0/0, scheduled: 0
loaded plugins: charon aesni random nonce x509 revocation constraints pubkey pkcs1 pkcs7 pkcs7 dnskey sshkey pem openssl pkcs8 agent gcm drbg attr kernel-netlink resolve socket-default connmark stroke updown
chapv2 xauth-generic counters
Listening IP addresses:
    192.168.5.128
    172.17.0.1
Connections:
    myvpn: %any...%any IKEv2
    myvpn: local: [server] uses public key authentication
    myvpn: remote: uses EAP_MSCHAPV2 authentication
    myvpn: child: 0.0.0.0/0 == dynamic TUNNEL
Security Associations (0 up, 0 connecting):
    none

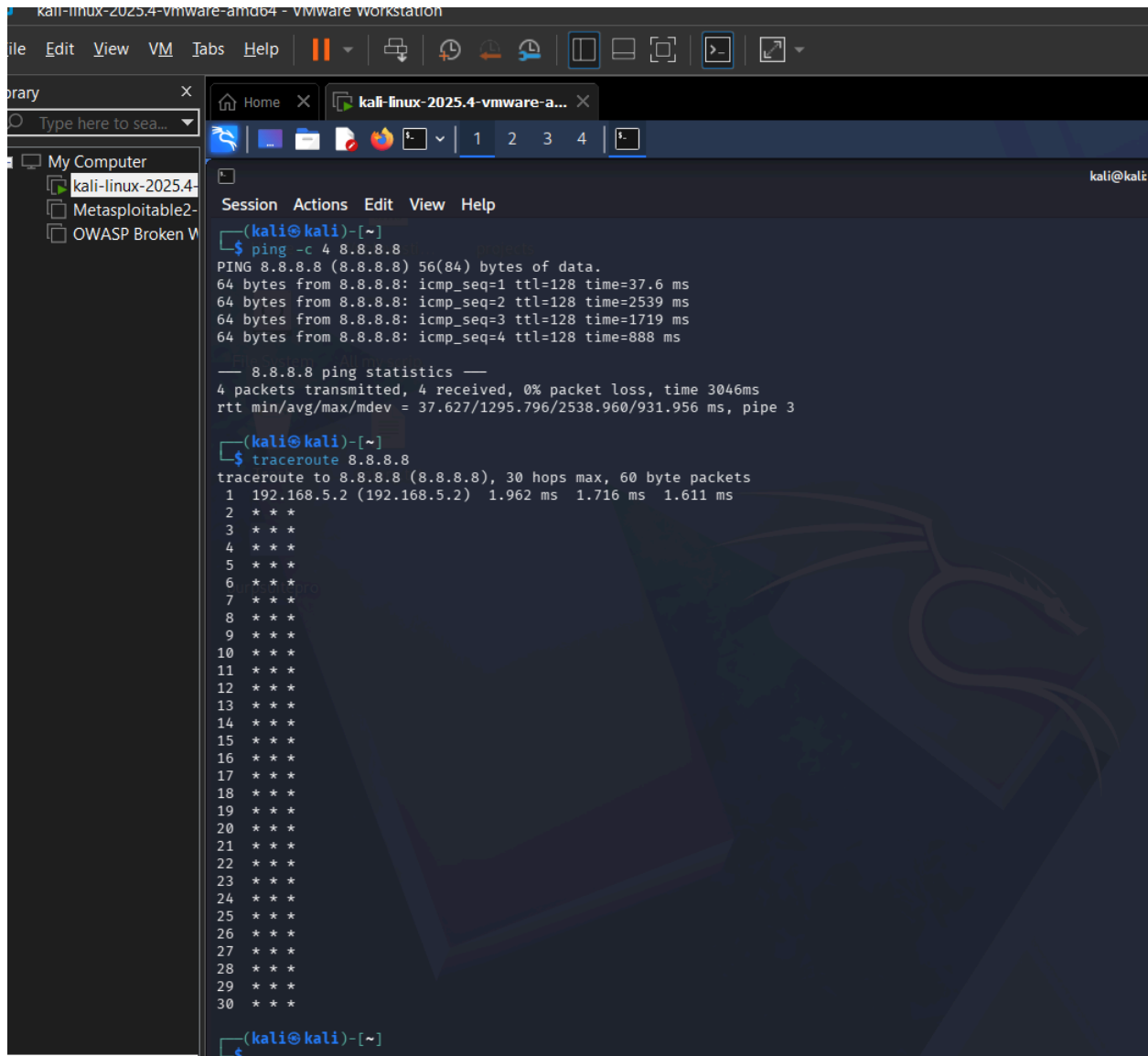
(kali@kali)-[~]
$
```

The IPsec configuration was verified successfully. The myvpn connection was loaded correctly, and StrongSwan recognized the IKEv2 configuration. No active Security Associations were present because no VPN connection had been established.

Step 5: Test the VPN connection (ping/trace route)

Commands Used:

```
ping -c 4 8.8.8.8
traceroute 8.8.8.8
```



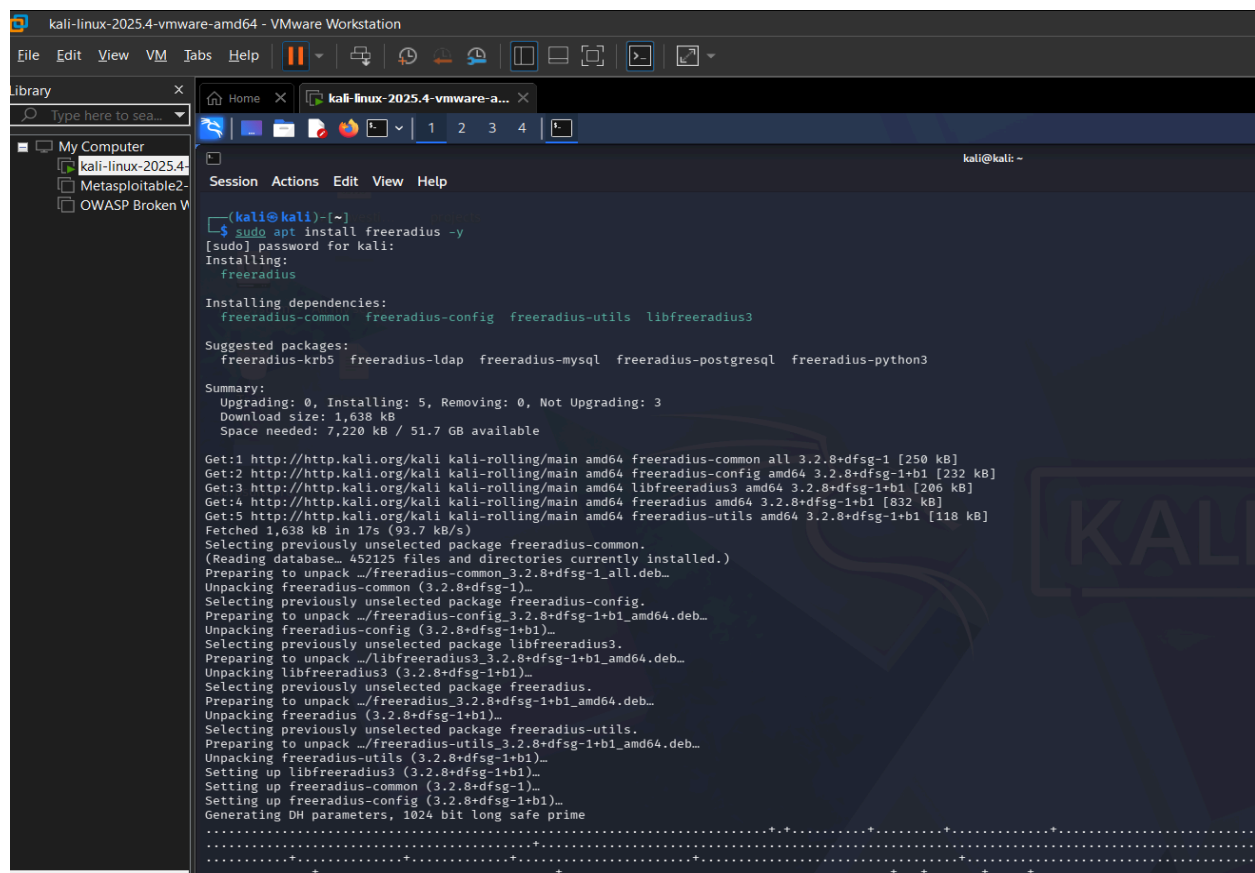
The screenshot shows a Kali Linux terminal window with the following output:

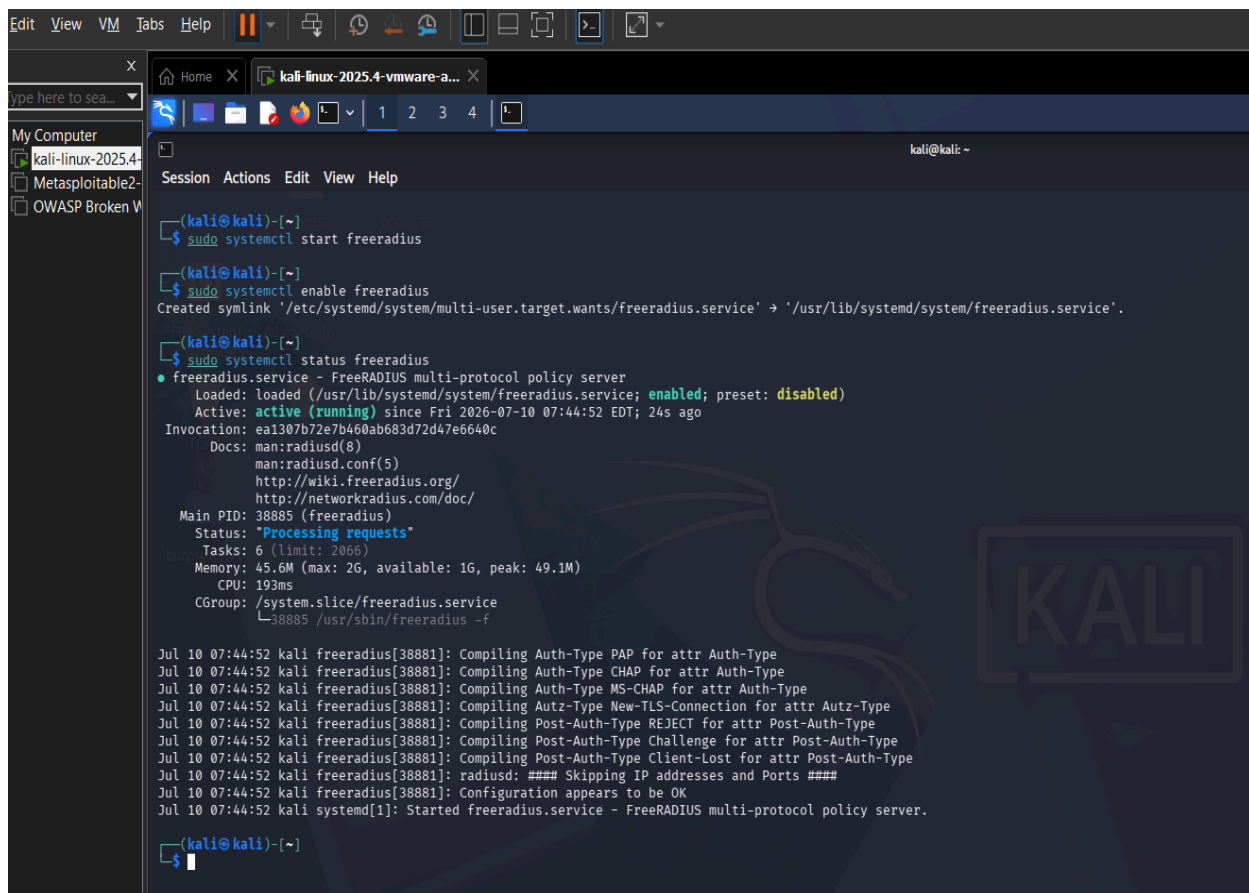
```
(kali@kali)-[~]
$ ping -c 4 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=37.6 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=2539 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=1719 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=128 time=888 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3046ms
rtt min/avg/max/mdev = 37.627/1295.796/2538.960/931.956 ms, pipe 3

(kali@kali)-[~]
$ traceroute 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
 1  192.168.5.2 (192.168.5.2)  1.962 ms  1.716 ms  1.611 ms
 2  * * *
 3  * * *
 4  * * *
 5  * * *
 6  * * *
 7  * * *
 8  * * *
 9  * * *
10  * * *
11  * * *
12  * * *
13  * * *
14  * * *
15  * * *
16  * * *
17  * * *
18  * * *
19  * * *
20  * * *
21  * * *
22  * * *
23  * * *
24  * * *
25  * * *
26  * * *
27  * * *
28  * * *
29  * * *
30  * * *
```

The VPN verification was performed using ping and traceroute. The ping test showed successful connectivity with 0% packet loss. The traceroute reached the local gateway, while subsequent hops did not respond, which is common because many routers block traceroute requests.



A screenshot of a Kali Linux terminal window. The terminal shows the following commands and output:

```
(kali@kali)-[~]
$ sudo systemctl start freeradius
(kali@kali)-[~]
$ sudo systemctl enable freeradius
Created symlink '/etc/systemd/system/multi-user.target.wants/freeradius.service' → '/usr/lib/systemd/system/freeradius.service'.
(kali@kali)-[~]
$ sudo systemctl status freeradius
● freeradius.service - FreeRADIUS multi-protocol policy server
   Loaded: loaded (/usr/lib/systemd/system/freeradius.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-07-10 07:44:52 EDT; 24s ago
     Invocation: ea1307b72e7b460ab683d72d47e6640c
       Docs: man:radiusd(8)
            man:radiusd.conf(5)
            http://wiki.freeradius.org/
            http://networkradius.com/doc/
    Main PID: 38885 (freeradius)
      Status: "Processing requests"
     Tasks: 6 (limit: 2066)
    Memory: 45.6M (max: 2G, available: 1G, peak: 49.1M)
       CPU: 193ms
    CGroup: /system.slice/freeradius.service
            └─38885 /usr/sbin/freeradius -f

Jul 10 07:44:52 kali freeradius[38881]: Compiling Auth-Type PAP for attr Auth-Type
Jul 10 07:44:52 kali freeradius[38881]: Compiling Auth-Type CHAP for attr Auth-Type
Jul 10 07:44:52 kali freeradius[38881]: Compiling Auth-Type MS-CHAP for attr Auth-Type
Jul 10 07:44:52 kali freeradius[38881]: Compiling Autz-Type New-TLS-Connection for attr Autz-Type
Jul 10 07:44:52 kali freeradius[38881]: Compiling Post-Auth-Type REJECT for attr Post-Auth-Type
Jul 10 07:44:52 kali freeradius[38881]: Compiling Post-Auth-Type Challenge for attr Post-Auth-Type
Jul 10 07:44:52 kali freeradius[38881]: Compiling Post-Auth-Type Client-Lost for attr Post-Auth-Type
Jul 10 07:44:52 kali freeradius[38881]: radiusd: #### Skipping IP addresses and Ports ####
Jul 10 07:44:52 kali freeradius[38881]: Configuration appears to be OK
Jul 10 07:44:52 kali systemd[1]: Started freeradius.service - FreeRADIUS multi-protocol policy server.

(kali@kali)-[~]
$
```

The FreeRADIUS service was started and enabled successfully. Its status was verified, confirming that the RADIUS server was running.

Step 3: Test the RADIUS server.

Command Used:

`sudo freeradius -X` (starts FreeRADIUS in debug)

```
kali@kali: ~  
$ sudo freeradius -X  
FreeRADIUS Version 3.2.8  
Copyright (C) 1999-2025 The FreeRADIUS server project and contributors  
There is NO warranty; not even for MERCHANTABILITY or FITNESS FOR A  
PARTICULAR PURPOSE  
You may redistribute copies of FreeRADIUS under the terms of the  
GNU General Public License  
For more information about these matters, see the file named COPYRIGHT  
  
FreeRADIUS is developed, maintained, and supported by InkBridge Networks.  
For commercial support, please email sales@inkbridgenetworks.com  
https://inkbridgenetworks.com/  
Starting - reading configuration files ...  
including dictionary file /usr/share/freeradius/dictionary  
including dictionary file /usr/share/freeradius/dictionary.dhcp  
including dictionary file /usr/share/freeradius/dictionary.vqp  
including dictionary file /etc/freeradius/3.0/dictionary  
including configuration file /etc/freeradius/3.0/radiusd.conf  
including configuration file /etc/freeradius/3.0/proxy.conf  
  
# Loading authorize { ... }  
Compiling Auth-Type New-TLS-Connection for attr Auth-Type  
# Loading preacct { ... }  
# Loading accounting { ... }  
# Loading post-auth { ... }  
Compiling Post-Auth-Type REJECT for attr Post-Auth-Type  
Compiling Post-Auth-Type Challenge for attr Post-Auth-Type  
Compiling Post-Auth-Type Client-Lost for attr Post-Auth-Type  
} # server default  
radiusd: ##### Opening IP addresses and Ports #####  
listen {  
    type = "auth"  
    ipaddr = 127.0.0.1  
    port = 18120  
Failed binding to auth address 127.0.0.1 port 18120 bound to server inner-tunnel: Address already in use  
/etc/freeradius/3.0/sites-enabled/inner-tunnel[33]: Error binding to port for 127.0.0.1 port 18120  
  
(kali@kali)~$ sudo systemctl status freeradius  
● freeradius.service - FreeRADIUS multi-protocol policy server  
   Loaded: loaded (/usr/lib/systemd/system/freeradius.service; enabled; preset: disabled)  
   Active: active (running) since Fri 2026-07-10 07:44:52 EDT; 13min ago  
     Invocation: ea1307b72e7b4e0ab683d72d47e640c  
       Docs: man:radiusd(8)  
            man:radiusd.conf(5)  
            http://wiki.freeradius.org/  
            http://networkradius.com/doc/  
    Main PID: 38885 (freeradius)  
      Status: "Processing requests"  
    Tasks: 6 (limit: 2066)  
  Memory: 45.6M (max: 2G, available: 1G, peak: 49.1M)  
     CPU: 204ms  
    CGroup: /system.slice/freeradius.service  
            └─38885 /usr/sbin/freeradius -f  
  
Jul 10 07:44:52 kali freeradius[38881]: Compiling Auth-Type PAP for attr Auth-Type  
Jul 10 07:44:52 kali freeradius[38881]: Compiling Auth-Type CHAP for attr Auth-Type  
Jul 10 07:44:52 kali freeradius[38881]: Compiling Auth-Type MS-CHAP for attr Auth-Type  
Jul 10 07:44:52 kali freeradius[38881]: Compiling Auth-Type New-TLS-Connection for attr Auth-Type  
Jul 10 07:44:52 kali freeradius[38881]: Compiling Post-Auth-Type REJECT for attr Post-Auth-Type  
Jul 10 07:44:52 kali freeradius[38881]: Compiling Post-Auth-Type Challenge for attr Post-Auth-Type  
Jul 10 07:44:52 kali freeradius[38881]: Compiling Post-Auth-Type Client-Lost for attr Post-Auth-Type  
Jul 10 07:44:52 kali freeradius[38881]: radiusd: ##### Skipping IP addresses and Ports #####  
Jul 10 07:44:52 kali freeradius[38881]: Configuration appears to be OK  
Jul 10 07:44:52 kali systemd[1]: Started freeradius.service - FreeRADIUS multi-protocol policy server.
```

FreeRADIUS was tested in debug mode. The server reported a port conflict because the FreeRADIUS service was already running and using the required port.

Problem Encountered

freeradius -X failed with "Address already in use" because the FreeRADIUS service was already active.

Troubleshooting

The service status had already been verified with:
`sudo systemctl status freeradius`

This confirmed that FreeRADIUS was running. The port conflict was expected when attempting to start a second instance in debug mode.

Step 4: Verify 802.1X Configuration

Since this is a single Kali VM and not a real network with a managed switch and client devices, you **cannot fully test 802.1X authentication**. The important verification for this lab is that the RADIUS server is running.

Command:

`sudo systemctl status freeradius`

```
(kali@kali)-[~]
└─$ sudo systemctl status freeradius
● freeradius.service - FreeRADIUS multi-protocol policy server
   Loaded: loaded (/usr/lib/systemd/system/freeradius.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-07-10 07:44:52 EDT; 13min ago
     Invocation: ea1307b72e7b460ab683d72d47e6640c
       Docs: man:radiusd(8)
            man:radiusd.conf(5)
            http://wiki.freeradius.org/
            http://networkradius.com/doc/
   Main PID: 38885 (freeradius)
     Status: "Processing requests"
    Tasks: 6 (limit: 2066)
   Memory: 45.6M (max: 2G, available: 1G, peak: 49.1M)
      CPU: 204ms
   CGroup: /system.slice/freeradius.service
           └─38885 /usr/sbin/freeradius -f

Jul 10 07:44:52 kali freeradius[38881]: Compiling Auth-Type PAP for attr Auth-Type
Jul 10 07:44:52 kali freeradius[38881]: Compiling Auth-Type CHAP for attr Auth-Type
Jul 10 07:44:52 kali freeradius[38881]: Compiling Auth-Type MS-CHAP for attr Auth-Type
Jul 10 07:44:52 kali freeradius[38881]: Compiling Autz-Type New-TLS-Connection for attr Autz-Type
Jul 10 07:44:52 kali freeradius[38881]: Compiling Post-Auth-Type REJECT for attr Post-Auth-Type
Jul 10 07:44:52 kali freeradius[38881]: Compiling Post-Auth-Type Challenge for attr Post-Auth-Type
Jul 10 07:44:52 kali freeradius[38881]: Compiling Post-Auth-Type Client-Lost for attr Post-Auth-Type
Jul 10 07:44:52 kali freeradius[38881]: radiusd: #### Skipping IP addresses and Ports ####
Jul 10 07:44:52 kali freeradius[38881]: Configuration appears to be OK
Jul 10 07:44:52 kali systemd[1]: Started freeradius.service - FreeRADIUS multi-protocol policy server.

(kali@kali)-[~]
```

The FreeRADIUS service was verified to be active and running. Although a complete 802.1X authentication test requires a managed switch and client device, the successful operation of the RADIUS server confirms that the authentication service is ready to process requests.

Summary of Challenges and How They Were Resolved

During the lab, a few challenges were encountered while configuring the different security protocols. The first issue occurred during the SSH configuration when the server continued listening on port 22 after changing it to port 2222. This happened because the Port 2222 setting was commented out in the SSH configuration file. After uncommenting the line and restarting the SSH service, the new port became active.

Another challenge occurred while installing StrongSwan. The installation process became unresponsive, causing the package manager to remain locked and preventing other installations. The running apt process was identified, terminated safely, and the installation was restarted successfully.

A final issue was encountered while testing FreeRADIUS in debug mode. An "Address already in use" error was displayed because the FreeRADIUS service was already running and using the required port. This was confirmed by checking the service status, showing that the server was functioning correctly.

These challenges provided valuable troubleshooting experience and reinforced the importance of checking service configurations, verifying running processes, and confirming service status during network security deployments.

Conclusion

This lab provided valuable hands-on experience in implementing key network security protocols. SSL/TLS secured web traffic, SSH enabled secure remote administration, IPsec demonstrated secure VPN configuration, and FreeRADIUS

showed how authentication supports 802.1X network access control. The verification tests confirmed that the configurations were successful, while the troubleshooting process strengthened practical problem-solving skills. Overall, the lab improved my understanding of network security implementation and the importance of securing communication across networked systems.